Initial Paper Reproduction Report

Time Series Forecasting-Based Kubernetes Autoscaling Using Facebook Prophet and LSTM
(Guruge & Priyadarshana, *Frontiers in Computer Science*, 2025)

Saad Azhar (i230822)
Ali Ramish (i230651)

Cloud Computing Research — Cutoff 2 Group 35 Supervisor: Basharat Hussain
FAST NUCES Islamabad

October 2026

Abstract

We reproduce the core offline forecasting experiment of Guruge and Priyadarshana [1], which proposes a Prophet–LSTM hybrid model to forecast HTTP request rate for proactive Kubernetes autoscaling. Using the authors' public repository, data and hyperparameters, we reproduce their headline NASA result closely (MSE 64.00 vs. 63.84 reported; R^2 0.900 vs. 0.900) and their FIFA result within the same order of magnitude (MSE 1.18×10^{-5} vs. 1.1×10^{-5}; R^2 0.9982 vs. 0.9983). A code audit then shows that the released evaluation writes each one-step-ahead residual forecast into the row of the *current* minute, so the reported hybrid forecast at time t effectively contains the observed value at time t. When the identical predictions are evaluated at the timestamp they actually forecast, NASA MSE rises from 64.0 to 203.9 (R^2 0.900 $\rightarrow$ 0.681), which is worse than the paper's own Bi-LSTM baseline (183.6). On FIFA the aligned hybrid is twice as bad as a naive "same as last minute" forecast. We document datasets, environment, deviations and discrepancies, and use these findings to refine the research gap from our Cutoff 1 proposal: forecasting gains must be measured as timely, causal decisions inside a CPU-limited control loop rather than as offline accuracy.

Contents

1 Paper Details **3**

2 Understanding the Original Paper **3**
2.1 Problem addressed and importance . . . 3
2.2 Proposed approach . . . 3
2.3 System architecture . . . 3
2.4 Cloud computing component . . . 3
2.5 Multimodal component . . . 4
2.6 Experimental settings and hyperparameters . . . 4

3 Datasets **4**

4 Reproducibility Audit **5**

5 Environment and Configuration **5**

6 Reproduced Results **6**
6.1 Main comparison . . . 6
6.2 Explanation of differences . . . 6

7 Code Audit Findings **6**
7.1 F1 (critical): forecasts evaluated against the wrong timestamp 6
7.2 Other findings 7

8 What Could Not Be Reproduced **8**

9 Problems Encountered **8**

10 Limitations and Research Gap **8**
10.1 Limitations of the baseline 8
10.2 Refined research gap (for mentor discussion) 8

1 Paper Details

Title	Time series forecasting-based Kubernetes autoscaling using Facebook Prophet and Long Short-Term Memory
Authors	Pasan Bhanu Guruge (University of Westminster); Y. H. P. P. Priyadarshana (Informatics Institute of Technology, Colombo)
Venue	*Frontiers in Computer Science*, vol. 7, Art. 1509165 (Software section, original research, open access, CC BY)
Year	2025 (received 10 Oct 2024, accepted 29 Jan 2025, published 19 Feb 2025)
DOI / paper link	10.3389/fcomp.2025.1509165
Official code	`https://github.com/pasanbhanu/time-series-forcasting-with-facebook-prophet-lstm-hybrid-model` (commit `354f004`, 19 Feb 2025)
Related repos	Data preprocessing: `https://github.com/PasanBhanu/time-series-forcasting-benchmark-dataset-preprocessing` KEDA integration: `https://github.com/PasanBhanu/proactive-autoscaler-for-keda-kubernetes`
Our code	`https://github.com/Saadx-git/Cloud_project_I230822_I230651` (`reproduce.py`, logs, results JSON)

2 Understanding the Original Paper

2.1 Problem addressed and importance

The default Kubernetes Horizontal Pod Autoscaler (HPA) is *reactive*: it adds replicas only after CPU or memory utilisation has already crossed a threshold. Because new pods need time to become ready, sudden load increases cause service degradation, while conservative over-provisioning wastes resources. The paper argues for *proactive* autoscaling, in which the next minute's HTTP request rate is forecast and the replica count is set before demand arrives.

2.2 Proposed approach

A two-stage hybrid forecaster:

1. **Prophet** models trend and daily/weekly seasonality of requests-per-minute, producing $\hat{y}_t^P$.
2. The residual $r_t = y_t - \hat{y}_t^P$ is modelled by a **2-layer LSTM** (50 units each, dense output, look-back of one step), producing $\hat{r}_{t+1}$ from r_t.
3. The final forecast is $\hat{y}_{t+1} = \hat{y}_{t+1}^P + \hat{r}_{t+1}$.

The paper compares against ARIMA, Bi-LSTM, LSTM and two alternative hybrids (Prophet–Bi-LSTM, Prophet–GRU) using MSE, RMSE, MAE, R^2 and total prediction time (TPT).

2.3 System architecture

The autoscaler follows the Monitor–Analyse–Plan–Execute (MAPE) loop:

Monitor Prometheus scrapes the per-service HTTP request rate.

Analyse The hybrid model forecasts the request rate one minute ahead.

Plan The required replica count is the forecast divided by a per-pod rated capacity, with a scale-down ratio (RRS) to dampen removals (following Dang-Quang and Yoo [2]).

Execute The replica count is applied through the Kubernetes API.

2.4 Cloud computing component

The paper itself describes the Kubernetes integration only architecturally; **no live-cluster experiment is reported**. The companion KEDA repository provides a deployable implementation:

- a Python gRPC *KEDA External Scaler* that queries Prometheus for the 1-minute request rate of the service (metric `http_server_requests_seconds_count`), runs the hybrid model and returns the desired pod count as a custom metric;
- a KEDA `ScaledObject` that polls every 60 s with a 30 s cooldown, 1–10 replicas, `podLimit: 1000` requests per pod and a `scaleFactor` of 100;
- a Spring Boot sample API exporting Prometheus metrics, a Dockerfile and a Flask dashboard storing decisions in SQLite.

The *Autoscaler Evaluation* notebook compares the hybrid-based autoscaler against an HPA *simulated in Python* on a 400-minute FIFA slice, using under/over-provisioning metrics ($\Theta_U, \Theta_O, T_U, T_O$) and an elastic speed-up score.

2.5 Multimodal component

Not applicable. The input is a single univariate time series (requests per minute). The cloud-autoscaling topic was approved for this course; we note this explicitly because the course guidelines prefer multimodal data.

2.6 Experimental settings and hyperparameters

Table 1: Hyperparameters: paper (Table 2) versus released code.

Parameter	Paper	Released code	Note
Prophet growth	linear	linear (NASA), **flat** (FIFA)	FIFA differs
Changepoint prior scale	5.1	5.1	
Seasonality prior scale	30	30	
Yearly / weekly / daily seasonality	off / on / on	off / Fourier 20 / Fourier 50	orders not in paper
LSTM layers / units	2 / 50	2 / 50	
Look-back window	not stated	1 step	critical, see §7
Optimiser / LR	Adam / 0.001	Adam (default 0.001)	
Batch size	16	16	
Epochs / early stopping	50 / patience 5	**20** / patience 5	epochs differ
Train/test split	70/30 chronological	70/30, `shuffle=False`	no validation set for Prophet
Missing values	replaced with zero	forward-fill (train only)	differs; no NaNs present
Hyperparameter tuning	Keras-Tuner ranges (Table 3)	disabled (tuning flag set to False)	
Random seed	not stated	42 (NumPy, TF)	

Hardware, assumptions and acknowledged limitations. The paper reports no hardware, cloud resources, operating system or library versions, and no run-to-run variance. Stated limitations: the hybrid model has higher prediction time than single models; on FIFA its R^2 is slightly below Bi-LSTM; only horizontal scaling and request rate are considered. Future work proposed: reducing prediction time, implementing the MAPE loop in Kubernetes/KEDA, and vertical scaling. Implicit assumptions: one-minute control interval, linear relationship between request rate and required pods, and a fixed per-pod capacity.

3 Datasets

The preprocessed CSV files ship with the repository, so the raw logs did not have to be re-processed. We verified that the files in the main repository are byte-identical in size to those in the preprocessing

Table 2: Datasets used in the paper and in this reproduction.

	NASA-HTTP	FIFA World Cup 1998
Source	NASA Kennedy Space Center WWW logs, Internet Traffic Archive (`https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html`)	WorldCup98 access logs, Internet Traffic Archive (`https://ita.ee.lbl.gov/html/contrib/WorldCup.html`)
Period	1 Jul – 31 Aug 1995	30 Apr – 26 Jul 1998
Raw size (paper)	3,461,612 requests	1,352,804,107 requests
Released file	`datasets/nasa.csv`, 81,396 rows (1.9 MB)	`datasets/fifa.csv`, 125,300 rows (3.2 MB)
Modality	univariate time series (requests/min)	univariate time series (requests/min)
Characteristics	strong daily and weekly seasonality; outage gap 28 Jul – 3 Aug 1995 (not filled)	long quiet periods with extreme match-day spikes
Split	70 % train (56,977) / 30 % test (24,419), chronological	70 % / 30 %, chronological
Preprocessing	log lines aggregated to per-minute counts; duplicates dropped	as NASA, plus Min–Max scaling to $[0, 1]$
Augmentation	none	none

repository (1,870,783 and 3,157,125 bytes). Modalities are not combined, as there is only one.

4 Reproducibility Audit

Table 3: Reproducibility checklist.

Item	Available	Comment
Official repository	Yes	Jupyter notebooks, CITATION.cff, licence
Source code (models)	Partial	Prophet–LSTM (NASA, FIFA), Prophet–BiLSTM, Prophet–GRU only. **No code** for the ARIMA, LSTM or Bi-LSTM baselines in Tables 4–5
Training/experiment scripts	Yes	notebooks; hyperparameter search included but disabled
Datasets	Yes	preprocessed CSVs included; raw sources public
Pre-trained models	Yes	Prophet JSON and Keras files, Sept 2024
Configuration files	Partial	hyperparameters hard-coded in notebooks
Pinned dependencies	No	unversioned `!pip install` calls only
Hardware/cloud requirements	No	not stated anywhere
Installation / run instructions	No	README contains abstract and links only
Instructions to reproduce tables	No	none; notebook outputs are cleared
Live Kubernetes experiment	Partial	KEDA scaler code and manifests exist, but no results or experiment protocol

5 Environment and Configuration

Deviations from the original experiment.

1. The notebooks were transcribed into one script, `reproduce.py`, keeping the cell logic and order but removing plotting code. Two adaptations were needed for current library versions: pandas 3 removed `fillna(method=...)` and chained `.iloc` assignment, so we use `.ffill()` and explicit Series construction with identical semantics. Keras 3 requires an explicit `Input` layer instead of `input_shape`.
2. Library versions are newer than those available to the authors in 2024 (theirs are unknown).
3. We did not re-run hyperparameter tuning; neither did the released code.
4. Only the headline Prophet–LSTM rows of Tables 4 and 5 were reproduced (see §8).

Table 4: Reproduction environment.

Hardware	Laptop, Intel Core i5-7300U (2 cores / 4 threads, 2.6 GHz), 15 GB RAM, no GPU
Cloud resources	none (all experiments are offline and CPU-only)
Operating system	Fedora Linux 43, kernel 7.2.4
Python	3.11.16 (isolated `uv` virtual environment)
Libraries	prophet 1.5.0 (cmdstanpy 1.3.0), tensorflow-cpu 2.21.0, keras 3.15.1, scikit-learn 1.9.1, pandas 3.0.6, numpy 2.4.6; full lock file in `requirements.lock.txt`
Seeds	42 (NumPy, TensorFlow), as in the notebooks
Configuration	identical to the released notebooks (Table 1, "Released code" column); LSTM trained for at most 20 epochs with early stopping

6 Reproduced Results

6.1 Main comparison

Table 5: Prophet–LSTM hybrid: original paper versus our reproduction (released evaluation code, unchanged logic). Difference = reproduced − original. FIFA values are on the Min–Max-scaled series, as in the paper.

Dataset	Metric	Original paper	Reproduced	Difference
NASA	MSE	63.836	63.997	+0.161 (+0.25 %)
	RMSE	7.990	8.000	+0.010
	MAE	6.164	6.178	+0.014
	R^2	0.900	0.900	0.000
	TPT (ms)	3,492	6,398	+2,906 (+83 %)
FIFA	MSE	0.000011	0.0000118	+0.0000008 (+7.7 %)
	RMSE	0.003358	0.003442	+0.000084 (+2.5 %)
	MAE	0.000675	0.000632	−0.000043 (−6.3 %)
	R^2	0.998318	0.998233	−0.000085
	TPT (ms)	3,971	7,326	+3,355 (+84 %)

6.2 Explanation of differences

- **Accuracy metrics** match within 0.3 % on NASA. The residual difference is consistent with different TensorFlow/Prophet versions (Stan optimiser, LSTM kernel numerics) and early stopping halting at a slightly different epoch (6 epochs on both datasets in our runs). On FIFA, MSE is 7.7 % higher and MAE 6.3 % lower; at the 10^{-5}–10^{-6} scale of the normalised series, these are tiny absolute differences (R^2 differs by 0.0001). We consider the NASA headline result *reproduced* and the FIFA result *reproduced*.
- **Total prediction time** is 83 % higher. TPT is the wall-clock time of one batched `predict()` call over all 24,419 test points (Prophet) plus one batched LSTM call. Our CPU is a 2017 dual-core laptop processor, and the authors' hardware is unknown, so absolute times are not comparable. TPT is also not the latency of a single autoscaling decision; it amortises to roughly 0.26 ms per forecast on our machine. This metric therefore says little about whether a decision is ready within a control interval, which matters for our research gap (§10).

7 Code Audit Findings

7.1 F1 (critical): forecasts evaluated against the wrong timestamp

The LSTM dataset is built so that sample i has input r_i and target r_{i+1}; the model output p_i is therefore a forecast of r_{i+1}. The released code then writes the predictions back as

```
final_df['residual_predict'].iloc[0:len(combine_predict)] = combine_predict[:, 0]
final_df['y_predicted'] = final_df['yhat'] + final_df['residual_predict']
```

which places p_i (the forecast for minute $i{+}1$) in row i. The evaluated "forecast" for minute i is therefore

$$\tilde{y}_i = \hat{y}_i^P + f(r_i), \qquad r_i = y_i - \hat{y}_i^P,$$

a function of the *observed* value y_i it is supposed to predict. With a one-step look-back, the LSTM learns $f(r) \approx r$, so $\tilde{y}_i \approx y_i$. In our run, the correlation between the released residual "prediction" and the same-row actual residual is **0.9997** on the NASA test set (0.9998 on FIFA).

To quantify the effect, we keep the trained models and predictions unchanged and only place p_i in row $i{+}1$, the minute it actually forecasts:

Table 6: Effect of correcting the alignment, compared with simple references on the same test rows. Paper baselines are copied from the paper's Tables 4–5 (no code released).

	NASA				FIFA (scaled)		
Model	MSE	RMSE	MAE	R^2	MSE	MAE	R^2
Hybrid, as released (paper)	63.84	7.99	6.16	0.900	1.1e-5	6.75e-4	0.9983
Hybrid, as released (ours)	64.00	8.00	6.18	0.900	1.18e-5	6.32e-4	0.9982
Hybrid, correctly aligned (ours)	**203.90**	**14.28**	**10.89**	**0.681**	**2.21e-5**	**1.89e-3**	**0.9967**
Prophet only (ours)	268.02	16.37	12.56	0.581	6.56e-3	3.95e-2	0.021
Persistence $\hat{y}_t = y_{t-1}$ (ours)	268.96	16.40	12.28	0.579	1.11e-5	1.63e-3	0.9984
Bi-LSTM 1-step (paper)	183.64	13.55	10.28	0.712	3.6e-5	3.13e-3	0.9986
ARIMA 1-step (paper)	196.29	14.01	10.57	0.692	4.0e-5	3.30e-3	0.9985

Consequence. Once aligned, the NASA hybrid still beats Prophet alone and naive persistence (MSE −24%), but it no longer beats the paper's own single-model baselines: its MSE is 11 % worse than Bi-LSTM and 4 % worse than ARIMA. The headline claim of a "65–90 % accuracy" improvement over single models therefore does not hold under a causal evaluation. On FIFA the aligned hybrid's MSE (2.21e-5) is **twice that of naive persistence** (1.11e-5), and MAE is three times the as-released value. The as-released FIFA score is almost identical to persistence, as expected when the "forecast" contains the current observation. Prophet alone explains almost none of the FIFA variance (R^2 0.02), so nearly all of the hybrid's apparent skill comes from the residual stage. Persistence also outperforms the paper's FIFA Bi-LSTM and ARIMA figures, which suggests those baselines were weak or evaluated differently; this cannot be checked without their code.

Scope note. The deployed KEDA scaler is *not* affected: it feeds the current residual r_t to the LSTM and adds the result to Prophet's forecast for $t{+}1$, which is correct. The defect is in the offline evaluation that produced the published tables.

7.2 Other findings

F2 Test-set leakage in scaling. The Min–Max scalers (NASA residuals; FIFA raw series) are fitted on train and test data together. The effect is small for 1-step forecasts but violates a held-out protocol.

F3 Paper–code mismatches. These are Prophet growth (flat for FIFA), Fourier orders, epochs (20 vs. 50) and missing-value handling (forward-fill vs. zero), listed in Table 1. The Autoscaler Evaluation notebook documents `RRS = 0.1` but uses `rrs = 0.6`.

F4 KEDA scaler extrapolates 30 years. The scaler queries Prophet at `datetime.now()`, but the model was trained on 1995 timestamps. With linear growth and a changepoint prior of 5.1, the trend term is extrapolated about 30 years. The residual scaler is also hard-coded to the range $[-118, 284]$ from the NASA training data, so live behaviour depends on traffic resembling NASA-1995 in scale.

F5 Prediction-time metric. TPT is a batch throughput measure over the whole test set. It is not per-decision latency and is reported without repetitions, and the CPU is unspecified.

F6 Missing baselines and HPA experiment. ARIMA, LSTM and Bi-LSTM baselines have no code. The HPA comparison exists only as a Python simulation on a 400-minute slice and is not reported in the paper.

8 What Could Not Be Reproduced

- **ARIMA, LSTM and Bi-LSTM baselines (Tables 4–5):** no code was released and the configurations are under-specified (ARIMA orders, input windows). We quote the paper's values and do not reimplement them in this cutoff.
- **Live Kubernetes behaviour:** the paper reports no live results, so there is nothing to reproduce. A cluster run is part of our next stage.
- **Run-to-run variance:** single seed only, matching the released code.

9 Problems Encountered

Installation No pinned versions. Notebook syntax (nested f-string quotes) requires Python $\geq$3.12 inside notebooks, and pandas 3/Keras 3 API changes required small edits. All edits are documented above.

Missing resources No README instructions, no hardware statement, notebook outputs cleared, baseline code absent.

Code problems Findings F1–F5.

Dataset problems None blocking; the preprocessed data is provided. The NASA outage gap is left in the series, so test-period seasonality includes a structural break.

Computational limits The LSTM trains at about 16 s per epoch and Prophet fits in about 2 min (NASA) or 34 s (FIFA, flat growth) on our 2-core CPU; the whole reproduction takes roughly 15 minutes.

10 Limitations and Research Gap

10.1 Limitations of the baseline

1. The accuracy advantage is evaluated offline, and the published evaluation is non-causal (F1). Corrected, the hybrid does not beat simpler single models.
2. The forecaster is never evaluated inside a running autoscaler. Service quality (latency SLO violations) and resource use (pod-seconds) are not measured.
3. Computational cost is reported as batch prediction time on unknown hardware. The cost of the controller itself (CPU, memory, per-decision latency against the 60 s KEDA polling interval) is not measured, although the authors list prediction-time reduction as future work.
4. There are no variance estimates, and a single train/test split is used for both datasets.

10.2 Refined research gap (for mentor discussion)

Our Cutoff 1 proposal asked whether the extra computation of hybrid forecasters pays off when the autoscaling controller has a limited CPU budget. The reproduction strengthens and sharpens this:

> *When predictive autoscalers are evaluated causally and end-to-end, inside a Kubernetes control loop whose controller CPU is constrained, does a complex hybrid forecaster (Prophet–LSTM) deliver better SLO compliance per pod-second than lightweight forecasters (persistence, seasonal-naive, linear regression) and a tuned HPA?*

This is defensible because (i) the baseline's accuracy advantage disappears under correct alignment, (ii) the authors themselves name prediction time and Kubernetes integration as open problems, and (iii) related work [4, 5, 6] covers predictor comparison and overhead but not the interaction between

controller CPU budget and forecaster complexity under a shared scaling policy. Novelty still needs to be confirmed against recent citing papers before the confirmatory experiments.

References

[1] P. B. Guruge and Y. H. P. P. Priyadarshana, "Time series forecasting-based Kubernetes autoscaling using Facebook Prophet and Long Short-Term Memory," *Frontiers in Computer Science*, vol. 7, Art. 1509165, 2025, doi:10.3389/fcomp.2025.1509165.

[2] N.-M. Dang-Quang and M. Yoo, "Deep learning-based autoscaling using bidirectional long short-term memory for Kubernetes," *Applied Sciences*, vol. 11, no. 9, Art. 3835, 2021.

[3] M. Nashaat, W. Moussa, R. Rizk and W. Saber, "Dynamic machine learning approach for workload prediction in cloud environments," *Scientific Reports*, vol. 16, Art. 10983, 2026.

[4] G. Lanciano, F. Galli, T. Cucinotta, D. Bacciu and A. Passarella, "Extending OpenStack Monasca for predictive elasticity control," *Big Data Mining and Analytics*, vol. 7, no. 2, pp. 315–339, 2024.

[5] M. Imdoukh, I. Ahmad and M. G. Alfailakawi, "Machine learning-based auto-scaling for containerized applications," *Neural Computing and Applications*, vol. 32, pp. 9745–9760, 2020.

[6] N. S. Joshi, R. Raghuwanshi, Y. M. Agarwal, B. Annappa and D. N. Sachin, "ARIMA-PID: container auto scaling based on predictive analysis and control theory," *Multimedia Tools and Applications*, vol. 83, pp. 26369–26386, 2024.